

Curso desarrollo de aplicaciones con tecnologías web
Módulo formativo 2: Programación web en el entorno servidor

Unidad formativa 1: Desarrollo de aplicaciones web en el entorno servidor

Lección 2: La orientación a objetos

Herencia

Modularidad

Genericidad y sobrecarga

Desarrollo orientado a objetos

Lenguajes de modelización en el desarrollo orientado a objetos

Herencia

La herencia es un principio fundamental de la programación orientada a objetos que permite a una clase heredar atributos y métodos de otra. Facilita la reutilización del código y la creación de jerarquías de clases. En PHP, esto se logra con la palabra clave `extends`.

1. Concepto de herencia. Superclases y subclasses

- **Superclase:** Clase base que contiene atributos y métodos comunes.
- **Subclase:** Clase derivada que hereda de la superclase y puede agregar o modificar funcionalidades.

Ejemplo básico de herencia:

```
class Vehiculo {
    public $marca;
    public $modelo;

    public function __construct($marca, $modelo) {
        $this->marca = $marca;
        $this->modelo = $modelo;
    }

    public function obtenerInfo() {
        return "Vehículo: {$this->marca} {$this->modelo}";
    }
}

// Subclase Coche que hereda de Vehiculo
class Coche extends Vehiculo {
    public $puertas;

    public function __construct($marca, $modelo, $puertas) {
        parent::__construct($marca, $modelo); // Llamar al constructor de la superclase
        $this->puertas = $puertas;
    }

    public function obtenerInfo() {
        return parent::obtenerInfo() . " con {$this->puertas} puertas.";
    }
}

$coche = new Coche("Toyota", "Corolla", 4);
echo $coche->obtenerInfo();
```

◆ `parent::__construct()` llama al constructor de Vehiculo, evitando repetir código.

2. Herencia múltiple

PHP no soporta herencia múltiple de forma directa como otros lenguajes (C++), pero permite simularla mediante **interfaces** y **traits**.

Ejemplo con trait:

```
trait GPS {  
    public function obtenerUbicacion() {  
        return "Ubicación actual: 40.7128° N, 74.0060° W";  
    }  
}  
  
class Moto extends Vehiculo {  
    use GPS; // Usa el trait GPS  
  
    public function tipoVehiculo() {  
        return "Este es una moto.";  
    }  
}  
  
$miMoto = new Moto("Yamaha", "YZF-R3");  
echo $miMoto->mostrarInfo() . "<br>";  
echo $miMoto->obtenerUbicacion();
```

- ◆ La clase Moto hereda de Vehiculo.
- ◆ Se usa un trait llamado GPS para simular la herencia múltiple, lo que permite que Moto tenga acceso a la función obtenerUbicacion().

3. Clases abstractas

Una clase abstracta es una clase que no puede ser instanciada directamente. Se utiliza como una plantilla para otras clases, obligando a las clases hijas a implementar métodos abstractos que se definen en la clase base.

- **Métodos abstractos:** Son métodos que se definen en la clase abstracta sin implementación, y deben ser implementados por las clases hijas.

Ejemplo:

```
abstract class Animal {
    public $nombre;

    public function __construct($nombre) {
        $this->nombre = $nombre;
    }

    // Método abstracto
    abstract public function hacerSonido();
}

class Perro extends Animal {
    public function hacerSonido() {
        return "Ladra";
    }
}

$perro = new Perro("Rex");
echo $perro->hacerSonido(); // "Ladra"
```

- ◆ Animal es una clase abstracta que no se puede instanciar directamente.
- ◆ La subclase Perro implementa el método hacerSonido() de la clase abstracta.

4. Tipos de herencia

En programación orientada a objetos, hay diferentes tipos de herencia, que permiten crear relaciones entre clases de diferentes maneras:

1. **Herencia simple:** Una clase hija hereda de una sola clase base.
2. **Herencia multinivel:** Una clase hija hereda de una clase padre, y luego otra clase hereda de esa hija.
3. **Herencia jerárquica:** Varias clases hijas heredan de la misma clase padre.

Ejemplo de herencia multinivel:

```
class Animal {  
    public function respirar() {  
        return "Respirando...";  
    }  
}  
  
class Mamifero extends Animal {  
    public function mamar() {  
        return "Amamantando...";  
    }  
}  
  
class Humano extends Mamifero {  
    public function pensar() {  
        return "Pensando...";  
    }  
}  
  
$humano = new Humano();  
echo $humano->respirar(); // "Respirando..."  
echo $humano->mamar();    // "Amamantando..."
```

◆ Humano hereda de Mamifero, y Mamifero hereda de Animal.

5. Polimorfismo y enlace dinámico (dynamic binding)

El **polimorfismo** es un concepto que permite que un mismo método se comporte de manera diferente según la clase de objeto que lo invoque. El **enlace dinámico** o **dynamic binding** es el proceso mediante el cual PHP determina en tiempo de ejecución qué método se debe llamar en función del objeto que lo invoca.

Ejemplo de polimorfismo:

```
class Animal {
    public function hacerSonido() {
        return "Hace un sonido genérico.";
    }
}

class Perro extends Animal {
    public function hacerSonido() {
        return "Ladra.";
    }
}

class Gato extends Animal {
    public function hacerSonido() {
        return "Miaa.";
    }
}

$animal1 = new Perro();
$animal2 = new Gato();

echo $animal1->hacerSonido(); // "Ladra."
echo $animal2->hacerSonido(); // "Miaa."
```

◆ El método `hacerSonido()` se comporta de manera diferente según si el objeto es de la clase `Perro` o `Gato`.

6. Directrices para el uso correcto de la herencia

- **Usar la herencia de manera lógica:** La herencia debe reflejar una relación “es un” (por ejemplo, un Perro es un Animal).
- **No abusar de la herencia:** Si no tiene sentido, es mejor usar composición en lugar de herencia, ya que esto puede llevar a una jerarquía de clases muy compleja.
- **Preferir la composición sobre la herencia:** En lugar de hacer que una clase herede de otra, considere incluir instancias de otras clases dentro de la clase.

Modularidad

La **modularidad** es un principio de diseño que consiste en dividir un programa en módulos independientes que pueden ser desarrollados, mantenidos y reutilizados de forma aislada. Esto mejora la organización del código y facilita su mantenimiento.

1. Librerías de clases. Ámbito de utilización de nombres

Las librerías de clases son colecciones de clases que realizan tareas específicas, como acceder a bases de datos, manejar archivos, o realizar operaciones matemáticas. Estas librerías pueden ser reutilizadas en diferentes proyectos.

- **Ámbito de utilización de nombres:** PHP utiliza espacios de nombres (namespaces) para organizar las clases y evitar conflictos de nombres entre ellas.

Ejemplo de librería de clases con namespaces:

```
namespace Vehiculos;
```

```
class Coche {
    public $marca;
    public $modelo;

    public function __construct($marca, $modelo) {
        $this->marca = $marca;
        $this->modelo = $modelo;
    }

    public function obtenerInfo() {
        return "Vehículo: {$this->marca} {$this->modelo}";
    }
}
```

```

namespace Usuarios;

class Usuario {
    public $nombre;

    public function __construct($nombre) {
        $this->nombre = $nombre;
    }

    public function obtenerNombre() {
        return $this->nombre;
    }
}

use Vehiculos\Coche;
use Usuarios\Usuario;

$coche = new Coche("Toyota", "Corolla");
$usuario = new Usuario("Juan");

echo $coche->obtenerInfo(); // "Vehículo: Toyota Corolla"
echo $usuario->obtenerNombre(); // "Juan"

```

- ◆ Se utilizan namespaces para organizar las clases Coche y Usuario en diferentes áreas (Vehículos y Usuarios).
- ◆ use se utiliza para importar las clases desde sus respectivos namespaces.

2. Ventajas de la utilización de módulos o paquetes

El uso de módulos o paquetes en programación ofrece varias ventajas:

1. **Reutilización de código:** Los módulos se pueden reutilizar en diferentes aplicaciones sin necesidad de reescribir el código.
2. **Mantenimiento más fácil:** Los módulos independientes facilitan el mantenimiento y la actualización de partes específicas del sistema sin afectar otras.
3. **Mejor organización del código:** Los módulos permiten estructurar el código de manera más ordenada, lo que facilita su comprensión y desarrollo.
4. **Colaboración entre equipos:** Los módulos permiten que diferentes equipos de trabajo trabajen en distintas partes del código sin interferencias.

Genericidad y sobrecarga

1. Concepto de genericidad

La **genericidad** es un concepto que permite crear funciones o clases que pueden trabajar con diferentes tipos de datos, sin necesidad de especificar un tipo concreto. En PHP, esto se puede lograr mediante tipos genéricos y plantillas (aunque PHP no tiene soporte completo para tipos genéricos como otros lenguajes).

Ejemplo de genericidad:

```
class Caja {
    private $contenido;

    public function agregarContenido($contenido) {
        $this->contenido = $contenido;
    }

    public function obtenerContenido() {
        return $this->contenido;
    }
}

$cajaEntero = new Caja();
$cajaEntero->agregarContenido(5); // Guardamos un número
echo $cajaEntero->obtenerContenido(); // 5

$cajaCadena = new Caja();
$cajaCadena->agregarContenido("Hola Mundo"); // Guardamos un string
echo $cajaCadena->obtenerContenido(); // "Hola Mundo"
```

◆ En este ejemplo, la clase Caja puede almacenar cualquier tipo de dato sin especificar un tipo concreto gracias a su flexibilidad.

2. Concepto de Sobrecarga. Tipos de sobrecarga

La sobrecarga es un concepto que permite definir varios métodos o funciones con el mismo nombre, pero con diferentes parámetros. Sin embargo, en PHP, la sobrecarga no es soportada de la misma forma que en otros lenguajes como Java, pero podemos simularla mediante el uso de métodos mágicos como `__call()`.

- **Sobrecarga de métodos:** Crear métodos con el mismo nombre pero con diferentes números o tipos de parámetros.

Ejemplo de sobrecarga en PHP:

```
class Calculadora {  
    public function __call($metodo, $argumentos) {  
        if ($metodo == "sumar") {  
            return array_sum($argumentos);  
        }  
    }  
}
```

```
$calc = new Calculadora();  
echo $calc->sumar(1, 2, 3); // 6  
echo $calc->sumar(10, 20); // 30
```

◆ En este ejemplo, el método `sumar` no está definido directamente en la clase, pero utilizamos `__call()` para simular la sobrecarga y sumar diferentes números de parámetros.

3. Comparación entre genericidad y sobrecarga

- **Genericidad:** Permite que un método o clase maneje diferentes tipos de datos sin especificar explícitamente el tipo.
- **Sobrecarga:** Permite definir múltiples versiones de un método o función, con diferentes números o tipos de parámetros, aunque PHP no tiene soporte nativo para la sobrecarga en el sentido tradicional.

Desarrollo orientado a objetos

1. Lenguajes de desarrollo orientado a objetos de uso común

Los lenguajes de programación orientados a objetos (OOP) más comunes incluyen:

- **Java:** Un lenguaje ampliamente utilizado para aplicaciones empresariales y móviles.
- **C++:** Usado para aplicaciones de alto rendimiento y sistemas embebidos.
- **Python:** Conocido por su sintaxis sencilla y usado en ciencia de datos, desarrollo web y automatización.
- **C#:** Utilizado principalmente en el desarrollo de aplicaciones de Windows y videojuegos.

2. Herramientas de desarrollo

Las herramientas más comunes para el desarrollo orientado a objetos incluyen:

- **Entornos de desarrollo integrados (IDEs)** como PHPStorm, Visual Studio Code, y Eclipse.
- **Frameworks** como Laravel (para PHP), Spring (para Java), Django (para Python) que proporcionan herramientas y bibliotecas para facilitar el desarrollo OOP.

Lenguajes de modelización en el desarrollo orientado a objetos

1. Uso del lenguaje unificado de modelado (UML) en el desarrollo orientado a objetos

El Lenguaje Unificado de Modelado (UML) es un estándar para visualizar, especificar, construir y documentar los artefactos de un sistema orientado a objetos. UML proporciona diagramas que permiten representar las clases, sus relaciones, y su comportamiento.

2. Diagramas para la modelización de sistemas orientados a objetos

Algunos diagramas importantes en UML incluyen:

1. **Diagrama de clases:** Representa las clases, atributos, métodos y sus relaciones.
2. **Diagrama de secuencia:** Muestra cómo los objetos interactúan entre sí a lo largo del tiempo.
3. **Diagrama de objetos:** Representa una instancia de clases y sus relaciones en un momento específico.